

Semana 7

RAG: Retrieval-Augmented Generation

Damos olhos ao cérebro

Semana 6: o LLM ganhou mãos (tool calling). Hoje: damos-lhe olhos — acesso a conhecimento que não está nos seus pesos.

Objectivos da aula

- 1 Compreender **porque** o LLM alucina — e porque fine-tuning não resolve
- 2 Entender **embeddings** como representações semânticas no espaço vectorial
- 3 Construir um **pipeline RAG** completo: ingest → chunk → embed → store → retrieve → generate
- 4 Explorar **chunking strategies** — e perceber porque importam mais do que parece
- 5 Lançar o **Mini-Project 2** (RAG pipeline, semanas 7–9)

O objectivo principal é compreender, não apenas construir.

Recap: mãos sem memória

Semana 5: o LLM é **stateless** — sem ferramentas, sem memória. Semana 6: demos-lhe **mãos** — structured output + tool calling.

Mas o LLM continua sem **olhos** para o conhecimento externo:

"Quem é o actual presidente da câmara de Lisboa?"
→ O modelo não sabe — os pesos foram treinados até 2023

"Qual a política de férias da minha empresa?"
→ O modelo não tem acesso — não está nos pesos

Tool calling resolve acções. RAG resolve conhecimento.

O problema: alucinação

Pergunta: "*Quantos presidentes teve o ISEL?*"

O ISEL teve 15 presidentes desde a sua fundação em 1852.
O primeiro presidente foi...

Parece credível. É correcto?

O modelo não "sabe" factos — gera tokens prováveis. Se o padrão linguístico for convincente, a resposta *parece* correcta mesmo quando não é.

Alucinação = o modelo gera conteúdo plausível mas factualmente errado, sem se aperceber.

Porque é que o LLM alucina?

O LLM é um modelo estatístico de linguagem:

$$p(x_t \mid x_{<t}) = f(x_{<t}, \theta)$$

O modelo optimiza **verosimilhança**, não **verdade**.

O modelo faz	O modelo NÃO faz
Prever o token mais provável	Verificar se é verdade
Produzir texto coerente	Aceder a fontes
Capturar padrões estatísticos	Distinguir facto de ficção

Um facto raro mas correcto é menos provável do que um facto comum mas errado.

THINK

A "alucinação" é um bug ou uma feature?

Pensem em dois cenários:

1. Assistente factual: "Quantos autarcas teve Lisboa?" → precisa de precisão
2. Escritor criativo: "Escreve um poema sobre Lisboa" → precisa de imaginação

*O mesmo mecanismo que produz alucinações (gerar o mais provável) é o que permite criatividade.
RAG não elimina a alucinação — dá ao modelo âncoras factuais para reduzir a probabilidade de erro.*

Pensem 1 minuto.

Fine-tuning não resolve

"Então treinamos o modelo com dados atualizados!"

Abordagem	Problema
Fine-tuning contínuo	Caro, lento, catastrophic forgetting
Context window infinita	Não existe; atenção dilui-se
Mais parâmetros	Ajuda, mas não resolve — não há pesos para toda a informação do mundo

O conhecimento do mundo é demasiado vasto e dinâmico para caber nos pesos.

Precisamos de dar ao modelo acesso a informação no momento em que precisa dela — não de a armazenar nos pesos.

A ideia do RAG

Retrieval-Augmented Generation (Lewis et al., 2020):

- 1 Receber a **pergunta** do utilizador
- 2 **Procurar** documentos relevantes numa base de conhecimento
- 3 **Injectar** os documentos no prompt
- 4 **Gerar** a resposta condicionada nos documentos

Sem RAG: pergunta → LLM → resposta (alucinação provável)

Com RAG: pergunta → retrieval → [docs] + pergunta → LLM → resposta (ancorada)

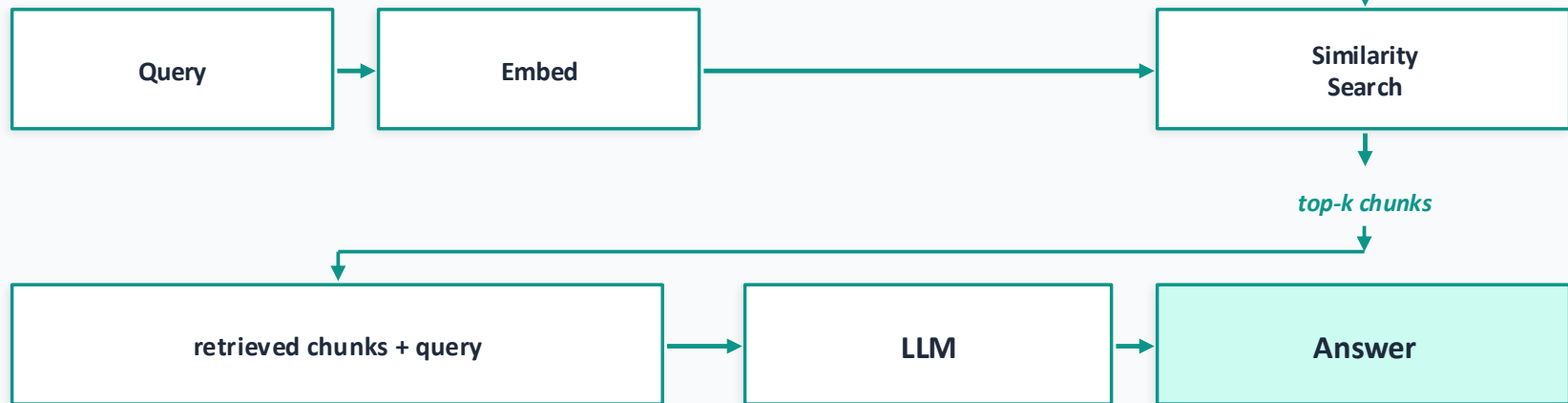
O modelo não precisa de "saber" — precisa de ler.

RAG – arquitetura geral

INDEXING (offline)



QUERY (online)



Seis passos: ingest → chunk → embed → store → retrieve → generate

Embeddings: de símbolos a vetores

Aula 03: **embeddings estáticos** (Word2Vec) captam relações:

king - man + woman $\approx$ queen

Mas a mesma palavra tem sempre o mesmo vector, mesmo em contextos diferentes.

Embeddings contextuais (BERT): o vector de "banco" muda conforme:

- "Sentou-se no **banco** do jardim" $\leftarrow$ banco de sentar
- "O **banco** aumentou as taxas" $\leftarrow$ banco financeiro

Hoje: um passo acima — embeddings de frases e documentos, necessários para RAG.

Encoder models: a família BERT

Na aula 03 distinguimos duas arquitecturas:

Decoder-only (GPT)	Encoder-only (BERT)
Causal attention ($\leftarrow$)	Full attention ($\leftrightarrow$)
Bom para gerar	Bom para compreender
Token seguinte	Representação semântica do todo
GPT-2, Llama, Mistral	BERT, MiniLM, MPNet

Para RAG, usamos **encoders** porque queremos **representações semânticas** — não geração.

Sentence-BERT / all-MiniLM: modelos encoder treinados para produzir embeddings onde frases com significado semelhante ficam próximas no espaço vectorial.

Similaridade semântica

Como medir se dois textos "significam a mesma coisa"?

Cosine similarity:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| \cdot |\mathbf{b}|}$$

Dois vectores na mesma direcção → similaridade ≈ 1 Vectores perpendiculares → similaridade ≈ 0

Texto A	Texto B	Cos. sim.
"Como instalar Ollama"	"Instalar o Ollama passo a passo"	0.91
"Como instalar Ollama"	"Receita de bolo de chocolate"	0.12

THINK

A cosine similarity é alta (≈ 0.87) entre:

- "O LLM alucina porque não tem acesso a fontes"
- "A IA é uma ameaça para a sociedade"

Alta similaridade semântica — mas o segundo documento responde à pergunta "porque o LLM alucina"?

*Recuperar documentos semelhantes não é o mesmo que recuperar documentos relevantes.
Esta diferença é crucial. Voltamos a ela na semana 8 (avaliação).*

Pensem 1 minuto.

Vector databases

Para procurar rapidamente entre milhões de vectores, precisamos de estruturas especializadas:

DB	Tipo	Quando usar
Chroma	Embedding DB simples	Protótipos, ensino
FAISS	Biblioteca (Meta)	Alta performance, milhões de vectores
Pinecone / Qdrant	Serviço cloud	Produção, escala

Operação central: **nearest neighbour search** — encontrar os k vectores mais próximos.

Busca exacta em 384 ou 768 dimensões é $O(n)$. Para milhões de vectores: **approximate nearest neighbour (ANN)** — sacrifica-se precisão marginal por ordens de magnitude de velocidade.

Chunking — o herói escondido

O chunking é provavelmente a decisão com **mais impacto** na qualidade do RAG — e a mais subestimada.

Porquê chunking?

- O documento inteiro é demasiado grande para o prompt
- Precisamos de granularidade: o parágrafo relevante, não o PDF de 200 páginas
- O embedding de um documento inteiro perde especificidade

Estratégia	Vantagem	Problema
Fixed-size (500 tokens)	Simples, determinístico	Corta contexto a meio
Sentence-level	Respeita fronteiras naturais	Chunks desiguais
Semantic	Agrupa por tópico	Mais complexo
Recursive + overlap	Contexto preservado	Redundância

Não há "melhor" estratégia — há a melhor para o vosso corpus.

O pipeline RAG completo

Seis passos, e cada um é uma decisão de design:

- 1 INGEST → Que documentos? Que formato? Que pré-processamento?
- 2 CHUNK → Que estratégia? Que tamanho? Que overlap?
- 3 EMBED → Que modelo? Que dimensão? EN ou multilingual?
- 4 STORE → Que DB? Que metadados? Que indexação?
- 5 RETRIEVE → Quantos resultados (k)? Que threshold?
- 6 GENERATE → Que prompt? Como formatar os chunks? Que modelo?

Cada passo tem trade-offs. A "arte" do RAG é escolher sabiamente em cada um.

Hands-on: construir um pipeline RAG

Vamos construir um pipeline RAG em **Python puro**, sem frameworks.

Stack mínimo:

- sentence-transformers (embeddings: paraphrase-multilingual-MiniLM-L12-v2, 384 dims)
- chromadb (vector database)
- ollama (geração local)

Corpus: CDRAM — 12 documentos fictícios sobre um clube desportivo, deliberadamente fora dos pesos do LLM (só assim testamos o retrieval).

Objectivo: fazer perguntas sobre o clube e obter respostas ancoradas nos documentos — não alucinadas.

Detalhes no guião. Trabalho em pares.

THINK

Antes de começar, pensem em três perguntas:

1. Se os chunks forem muito pequenos (1 frase), que problema aparece na resposta?
2. Se os chunks forem muito grandes (5 páginas), que problema aparece?
3. Se a pergunta for ambígua ("Como funciona o projecto?"), o que o retrieval vai devolver?

O pipeline é uma corrente — o elo mais fraco determina a qualidade.

Pensem 1 minuto.

RAG Pipeline

Peso: 15% · Entrega: semana 10 · Pipeline RAG sobre corpus à escolha

1

Pipeline funcional (código)

2

Relatório PDF (máx. 10 páginas): decisões, benchmark (10 perguntas), duas comparações predict → measure → explain, failure analysis

3

Discussão breve na aula

Framework: livre (Python puro, LlamaIndex, LangChain)

Stack recomendada: Ollama (nomic-embed-text + llama3.2:3b) + ChromaDB persistente — ver "Ponte para o MP2" no guião

AI Design Critic: prompt no Moodle — desafia decisões de design, não guia implementação

O que é avaliado: compreensão, não código perfeito. Decisões explicadas, não apenas implementadas. Falhas identificadas, não apenas sucessos.

Wrap-up

Hoje:

- O LLM alucina porque otimiza verosimilhança, não verdade
- RAG injecta conhecimento no momento da pergunta
- Embeddings representam semântica em espaço vectorial
- Chunking é a decisão com mais impacto — e a mais ignorada
- O pipeline: **ingest** → **chunk** → **embed** → **store** → **retrieve** → **generate**

A lição central:

O LLM ganhou "olhos" — pode ler documentos relevantes antes de responder. Mas ler não é compreender — a qualidade do RAG depende de cada passo do pipeline.

Semana 8 + TPC

TPC:

- Completar o pipeline RAG do guião — pelo menos 2 estratégias de chunking
- Comparar: mesma pergunta, chunks diferentes → resposta diferente?
- Registrar observações: onde o RAG falha e porque

Semana 8: RAG — avaliação e estratégias avançadas

- Como avaliar RAG: recall@k, faithfulness, relevance
- Comparação sistemática de estratégias de retrieval
- Técnicas avançadas: hybrid search, re-ranking, metadata filtering
- Quando o RAG falha — e o que fazer

De "funciona" a "funciona bem".